
◆ 1. Reverse Proxy

🔍 Summary:

An **Nginx reverse proxy** accepts client requests and forwards them to backend servers, hiding internal infrastructure and enabling load distribution, SSL, rate-limiting, etc.

✅ Config Example:

```
server {  
    listen 80;  
    server_name example.com;  
  
    location / {  
        proxy_pass http://localhost:5000;  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
    }  
}
```

💬 Interview Q&A:

Q1: What's the difference between a reverse proxy and a forward proxy? A1:

- **Reverse Proxy:** sits in front of servers, hides backend
 - **Forward Proxy:** sits in front of clients, hides user (e.g., proxy to access blocked sites)
-

Q2: Why use a reverse proxy like Nginx? A2:

- Security (expose only one point)
 - Central SSL termination
 - Load balancing
 - Path routing to different services
-

Q3: Can Nginx reverse proxy to HTTPS backends? A3:  Yes, just use `proxy_pass https://...` and configure `proxy_ssl_*` if needed.

Q4: How do you preserve the real client IP? A4: Use:

```
proxy_set_header X-Real-IP $remote_addr;
```

◆ 2. Load Balancing

Summary:

Nginx can load-balance traffic across multiple backend servers using **round-robin**, **least connections**, or **IP-hash**.

Config Example:

```
upstream backend {
    server backend1.local;
    server backend2.local;
}

server {
    location / {
        proxy_pass http://backend;
    }
}
```

Interview Q&A:

Q1: Load balancing methods supported by Nginx? A1:

- Round robin (default)
- Least connections
- IP-hash

- With Nginx Plus: **least time, hash with weights**
-

Q2: Can Nginx detect if a backend is down? A2:  Yes, with:

```
server backend1.local max_fails=3 fail_timeout=30s;
```

Q3: What's the difference between Nginx OSS and Nginx Plus for LB? A3:

- **Nginx OSS:** limited algorithms
 - **Plus:** active health checks, session persistence, dynamic updates
-

Q4: How to set weight per server? A4:

```
server backend1 weight=3;
server backend2 weight=1;
```

◆ 3. SSL Termination

Summary:

Nginx handles **SSL termination** by decrypting HTTPS requests at the proxy layer before passing plain HTTP to backend servers.

Config Example:

```
server {
    listen 443 ssl;
    ssl_certificate /etc/ssl/certs/site.crt;
    ssl_certificate_key /etc/ssl/private/site.key;

    location / {
        proxy_pass http://localhost:3000;
```

```
}  
}
```

Interview Q&A:

Q1: What is SSL termination? A1: The process of decrypting HTTPS traffic at the proxy, so backend servers handle only HTTP.

Q2: Can Nginx be both SSL terminator and passthrough? A2: Yes — you can decrypt some requests and tunnel others to upstream HTTPS endpoints.

Q3: How to redirect HTTP to HTTPS in Nginx? A3:

```
server {  
    listen 80;  
    return 301 https://$host$request_uri;  
}
```

Q4: How to enforce strong SSL/TLS settings? A4:

```
ssl_protocols TLSv1.2 TLSv1.3;  
ssl_prefer_server_ciphers on;  
ssl_ciphers HIGH:!aNULL:!MD5;
```

4. Caching

Summary:

Nginx can cache static files or proxy responses to improve performance and reduce backend load.

Config Example (Proxy Cache):

```
proxy_cache_path /tmp/nginx_cache levels=1:2 keys_zone=mycache:10m max_size=100m;

server {
    location /api/ {
        proxy_cache mycache;
        proxy_pass http://backend;
        proxy_cache_valid 200 1m;
    }
}
```

💬 Interview Q&A:

Q1: What can Nginx cache? A1:

- Static content (images, CSS, JS)
- Proxy responses (e.g., API data, rendered HTML)

Q2: What's `proxy_cache_use_stale` used for? A2: To serve stale cached responses during backend errors:

```
proxy_cache_use_stale error timeout updating;
```

Q3: Where is cache stored by default? A3: On disk (e.g., `/tmp/nginx_cache`) — needs explicit `proxy_cache_path`.

Q4: How to invalidate Nginx cache? A4:

- Manually delete cache files
- Use cache key versioning (`$arg_version`)
- Use `proxy_cache_bypass` or headers

◆ 5. HTTP Headers

Summary:

Nginx can **add, modify, or remove HTTP headers**, both in requests sent to upstream servers and responses sent to clients.

Config Example:

```
location / {  
    proxy_pass http://app;  
    proxy_set_header Host $host;  
    add_header X-Frame-Options "DENY";  
    add_header X-Content-Type-Options "nosniff";  
}
```

Interview Q&A:

Q1: What are common security-related headers to add? A1:

- `X-Frame-Options` : prevent clickjacking
 - `X-Content-Type-Options` : prevent MIME sniffing
 - `Content-Security-Policy` : control allowed content
 - `Strict-Transport-Security` : enforce HTTPS
-

Q2: Difference between `proxy_set_header` and `add_header` ? A2:

- `proxy_set_header` : for headers sent to **upstream servers**
 - `add_header` : for headers sent to **client/browser**
-

Q3: When does `add_header` not work? A3: If the response code isn't `200` , unless you add:

```
add_header Strict-Transport-Security "max-age=31536000" always;
```

6. Logging

Summary:

Nginx provides **access logs** (requests) and **error logs** (failures) which can be customized and used for observability, audits, and debugging.

Example:

```
access_log /var/log/nginx/access.log combined;
error_log /var/log/nginx/error.log warn;
```

Custom log format:

```
log_format custom '$remote_addr - $request_time "$request" $status';
access_log /var/log/nginx/access.log custom;
```

Interview Q&A:

Q1: What's the difference between access log and error log? A1:

- Access log: HTTP requests handled
 - Error log: Warnings, timeouts, upstream errors, etc.
-

Q2: How to log request time? A2: Use `$request_time` in a custom `log_format`.

Q3: Can logs be disabled? A3:  Yes:

```
access_log off;
```

Q4: What log levels does Nginx support? A4: `debug`, `info`, `notice`, `warn`, `error`, `crit`, `alert`, `emerg`

◆ 7. Upstreams

Summary:

Upstreams define groups of backend servers used for **load balancing**. You can configure health checks, failover, and weighting.

Example:

```
upstream api_backend {  
    server backend1.local;  
    server backend2.local backup;  
}
```

Interview Q&A:

Q1: What does `backup` mean in an upstream? A1: That server is used **only** if primary servers are down.

Q2: How do you route traffic to an upstream? A2:

```
proxy_pass http://api_backend;
```

Q3: Can you mix IP and DNS names in upstream? A3:  Yes, but DNS names are resolved **only at startup** unless you enable `resolver`.

Q4: How to configure max failures? A4:

```
server backend1.local max_fails=3 fail_timeout=10s;
```

8. Rate Limiting

Summary:

Rate limiting in Nginx is done using **leaky bucket** algorithms via `limit_req_zone` and `limit_req`.

Config Example:

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;

server {
    location /api/ {
        limit_req zone=api_limit burst=5 nodelay;
        proxy_pass http://backend;
    }
}
```

Interview Q&A:

Q1: How does Nginx rate limiting work? A1: It uses shared memory zones to track request rate and applies a leaky bucket mechanism.

Q2: What's the `burst` parameter for? A2: Allows short spikes before enforcing rate — a queue of extra requests beyond the rate limit.

Q3: What does `nodelay` do? A3: Rejects excess requests immediately instead of delaying them.

Q4: How to limit requests per IP? A4: Use `$binary_remote_addr` as the key in `limit_req_zone`.

9. Failover

Summary:

Nginx supports **failover** between backend servers using **upstream directives** like `max_fails`, `fail_timeout`, and `backup`.

✓ Config Example:

```
upstream myapp {  
    server app1.local max_fails=3 fail_timeout=10s;  
    server app2.local backup;  
}  
  
location / {  
    proxy_pass http://myapp;  
}
```

💬 Interview Q&A:

Q1: How does Nginx detect a failed upstream server? A1: Based on connection failure, timeout, or invalid response (e.g., HTTP 502) — tracked via `max_fails` and `fail_timeout`.

Q2: What does `backup` do in upstream block? A2: Marks a server to only be used when all primary servers are marked as failed.

Q3: Does Nginx perform active health checks? A3: ❌ Not in OSS. Only **Nginx Plus** supports active health checks; open source uses passive checks.

Q4: Can failover happen instantly? A4: No — it happens after `max_fails` are reached within `fail_timeout`.

◆ 10. Static Content

🔍 Summary:

Nginx is highly efficient at serving **static files** (HTML, images, JS, CSS) directly from the filesystem, making it ideal for CDN or frontend hosting.

✓ Config Example:

```
location /static/ {  
    root /var/www/html;  
    expires 30d;  
    access_log off;  
}
```

💬 Interview Q&A:

Q1: How does `root` differ from `alias` ? A1:

- `root` : appends URI to the path
- `alias` : replaces entire URI with path E.g.:

```
location /images/ {  
    root /data;      # /data/images/logo.png  
}  
location /images/ {  
    alias /data/;    # /data/logo.png  
}
```

Q2: How to cache static content in browser? A2:

```
expires 1y;  
add_header Cache-Control "public";
```

Q3: Should logs be enabled for static assets? A3: Not usually — use `access_log off;` for performance.

◆ 11. Security Headers

Summary:

Security headers instruct browsers how to handle content and protect users from XSS, clickjacking, and other attacks.

Config Example:

```
add_header X-Frame-Options "SAMEORIGIN";
add_header X-Content-Type-Options "nosniff";
add_header Referrer-Policy "no-referrer-when-downgrade";
add_header Content-Security-Policy "default-src 'self'";
```

Interview Q&A:

Q1: What does `X-Frame-Options` do? A1: Prevents the site from being embedded in `<iframe>` , blocking clickjacking attacks.

Q2: How to block MIME sniffing? A2:

```
add_header X-Content-Type-Options "nosniff";
```

Q3: What's CSP and why is it powerful? A3: Content-Security-Policy restricts the domains from which JS, fonts, styles can be loaded — helps block XSS and supply chain attacks.

12. Proxy Buffers

Summary:

Nginx uses **buffers** to temporarily store proxied responses. Proper tuning avoids memory bloat or “upstream too big” errors.

✓ Config Example:

```
proxy_buffer_size 16k;  
proxy_buffers 8 16k;  
proxy_busy_buffers_size 32k;
```

💬 Interview Q&A:

Q1: Why do buffer-related errors occur? A1: If the upstream response size exceeds buffer limits (especially for large headers), Nginx may log: `upstream sent too big header` .

Q2: When should you increase proxy buffer size? A2:

- Large response headers (e.g., big cookies)
 - Streaming large JSON/XML
 - Heavy reverse proxy usage
-

Q3: What's `proxy_buffers` vs `proxy_buffer_size` ? A3:

- `proxy_buffer_size` : for first chunk (headers)
 - `proxy_buffers` : for rest of the body
-
-

◆ 13. Logging Config

🔍 Summary:

Nginx logging can be **customized** using `log_format` , routed to multiple destinations, or even turned off per location.

✓ Config Example:

```
log_format main '$remote_addr - $remote_user [$time_local] "$request" '
                '$status $body_bytes_sent "$http_referer" '
                '"$http_user_agent" "$request_time"';

access_log /var/log/nginx/access.log main;
error_log /var/log/nginx/error.log warn;
```

Interview Q&A:

Q1: Can you have different logs per location? A1:  Yes — `access_log` can be overridden inside `server` or `location` blocks.

Q2: How to disable access log for health checks? A2:

```
location /health {
    access_log off;
    return 200 'OK';
}
```

Q3: What's `$request_time` ? A3: Time (in seconds) to process the request from client to Nginx (not backend time).

◆ 14. Metrics Monitoring

Summary:

Nginx does **not** expose Prometheus-style metrics natively. It does expose a **stub status** page with basic stats (connections, handled, requests).

Stub Status Config:

```
location /nginx_status {
    stub_status;
```

```
allow 127.0.0.1;
deny all;
}
```

Output:

```
Active connections: 5
accepts handled requests
1000 1000 1050
Reading: 0 Writing: 2 Waiting: 3
```

Interview Q&A:

Q1: What can be monitored from `stub_status` ? A1:

- Active connections
- Handled requests
- Connection states (read/write/wait)

Q2: How do you export Nginx metrics to Prometheus? A2: Use third-party exporters like `nginx-prometheus-exporter` .

Q3: Can Nginx send logs to ELK/Grafana? A3:  Yes — via Filebeat, Fluent Bit, or log drivers (e.g., `syslog`).

15. Nginx Reload

Summary:

Use `nginx -s reload` or `systemctl reload nginx` to apply config changes without downtime.

Commands:

```
nginx -t          # validate config
nginx -s reload    # send reload signal
```

Or:

```
systemctl reload nginx
```

Interview Q&A:

Q1: What's the difference between restart and reload? A1:

- **reload**: graceful, zero downtime
- **restart**: stops and starts Nginx — could drop connections

Q2: How does Nginx apply reload? A2:

- Spawns new workers with updated config
- Old workers gracefully shut down after completing requests

Q3: Should you always run `nginx -t` before reload? A3:  Yes — to ensure syntax is valid before applying.

16. Load Config

Summary:

Nginx config is **modular** — configs are loaded from main `nginx.conf`, which can include multiple files and directories.

Example:


```
http {  
    include /etc/nginx/conf.d/*.conf;  
    include /etc/nginx/sites-enabled/*;  
}
```

Interview Q&A:

Q1: What's the order of config loading? A1:

- `/etc/nginx/nginx.conf` is the main entry
- Includes in `http`, `events`, or `stream` blocks pull in partials

Q2: How to test Nginx config structure without restarting? A2:

```
nginx -t
```

Q3: What's the purpose of `sites-available` and `sites-enabled`? A3:

- `sites-available/` = all configs
- `sites-enabled/` = symlinks to active ones (manual activation)

Q4: What happens if one included config file is broken? A4: The whole reload/restart will fail unless you fix or remove that file — `nginx -t` will show the error.

◆ 17. Sticky Sessions

Summary:

Sticky sessions (a.k.a. session persistence) ensure that a user is routed to the same backend server across multiple requests — useful for session-based apps.

✓ Nginx OSS Example (using `ip_hash`):

```
upstream backend {  
    ip_hash;  
    server app1.local;  
    server app2.local;  
}
```

🔒 In Nginx Plus, more advanced options like `sticky cookie` or `sticky route` are available.

💬 Interview Q&A:

Q1: Why are sticky sessions useful? A1: To keep stateful user sessions (e.g., login tokens, cart items) bound to the same backend — especially if no shared cache like Redis is used.

Q2: What are the downsides of sticky sessions? A2:

- Poor load distribution
 - No automatic failover
 - Tied to client IP (e.g., `ip_hash`), which can be unreliable behind NAT
-

Q3: Is sticky session advisable in stateless microservices? A3: ❌ No — instead, centralize session store (e.g., Redis) or use JWT.

◆ 18. Logging Paths

🔍 Summary:

Nginx allows logs to be directed to **different paths** per environment, server block, or location. This helps organize logs for services or tenants.

✓ Example:

```
server {  
    access_log /var/log/nginx/site-a.log;  
}  
  
location /admin/ {  
    access_log /var/log/nginx/site-a-admin.log;  
}
```

Interview Q&A:

Q1: Why use different logging paths per location? A1:

- Debug specific API or path
- Isolate tenant logs
- Reduce noise in main logs

Q2: How do you log to stdout/stderr in Docker? A2:

```
access_log /dev/stdout;  
error_log /dev/stderr warn;
```

Q3: Can logs be sent to syslog? A3:  Yes:

```
access_log syslog:server=localhost:514;
```

19. Metrics Export

Summary:

To export Nginx metrics for **Prometheus/Grafana**, use tools like:

- `nginx-prometheus-exporter`
- `openresty/metrics`

- Custom exporters that parse logs

✅ Prometheus Exporter Setup:

```
nginx-prometheus-exporter -nginx.scrape-uri http://localhost/nginx_status
```

💬 Interview Q&A:

Q1: What metrics does exporter expose? A1:

- Active connections
 - Request rate
 - Response codes (2xx/4xx/5xx)
 - Upstream response time (if configured)
-

Q2: Can you add custom metrics from access logs? A2: ✅ Yes — parse logs with tools like:

- Promtail (Loki)
 - Vector.dev
 - Fluent Bit with Prom metrics plugin
-

Q3: Why doesn't Nginx natively support Prometheus? A3: Simplicity & performance — metrics are delegated to log analysis and third-party tools.

◆ 20. Rate Control

🔍 Summary:

Rate control lets you limit **how fast** data is sent to clients (e.g., video streaming or large file download throttling).

✓ Example:

```
location /download/ {  
    limit_rate 500k;  
    proxy_pass http://backend;  
}
```

Also available per connection:

```
limit_conn_zone $binary_remote_addr zone=addr:10m;  
limit_conn addr 10;
```

💡 Interview Q&A:

Q1: What's the difference between `limit_req` and `limit_rate` ? A1:

- `limit_req` : limits requests per second
- `limit_rate` : limits download speed per client

Q2: Can rate control help prevent abuse or DoS? A2: ✓ Yes — it slows down bots or bulk scrapers without blocking legit users.

Q3: How to allow burst but control long-term rate? A3: Use:

```
limit_req zone=my_zone burst=20 delay=5;
```

🎁 Bonus Nginx Topics (Advanced & Interview-Favorite)

◆ 1. GeoIP-Based Routing

🔍 Summary:

Serve different content or redirect users based on their IP's geographic location — great for **CDN**, **compliance**, or **multi-region** setups.

✓ **Config Example:**

```
geoip_country /usr/share/GeoIP/GeoIP.dat;

map $geoip_country_code $redirect_url {
    default      https://global.example.com;
    IN           https://in.example.com;
    US           https://us.example.com;
}

server {
    return 302 $redirect_url;
}
```

💬 **Interview Q:**

Q: How do you route users based on country in Nginx? **A:** Use the `geoip_country` directive with a MaxMind database, then use `$geoip_country_code` to map destinations.

◆ 2. Client Body Size Limits

🔍 **Summary:**

Prevents abuse by restricting **upload sizes** — essential for protecting from large file DoS or misbehaving clients.

✓ **Config:**

```
client_max_body_size 10M;
```

Use in:

- `http`

- `server`
 - `location` block
-

💬 Interview Q:

Q: How to prevent large POST uploads from overwhelming Nginx? A: Set `client_max_body_size` to limit payloads. Nginx will return 413 if exceeded.

◆ 3. Connection Timeouts (Keepalive, Read/Write)

🔍 Summary:

Control how long Nginx waits on slow or idle clients and backends — critical for **resource management** in high-load systems.

✅ Config Example:

```
keepalive_timeout 65;
client_body_timeout 15;
client_header_timeout 15;
send_timeout 10;
proxy_read_timeout 30;
```

💬 Interview Q:

Q: What's the difference between `keepalive_timeout` and `proxy_read_timeout`? A:
`keepalive_timeout` = idle time before closing client connection; `proxy_read_timeout` = how long to wait for upstream to respond.

◆ 4. WebSocket Proxying

🔍 Summary:

Required for apps using **WebSockets** (e.g., chat, live data). You must explicitly set headers to support `Upgrade` connections.

✓ **Config:**

```
location /ws/ {  
    proxy_pass http://localhost:8080;  
    proxy_http_version 1.1;  
    proxy_set_header Upgrade $http_upgrade;  
    proxy_set_header Connection "upgrade";  
}
```

💬 **Interview Q:**

Q: What changes are needed in Nginx to support WebSockets? A: Use `proxy_http_version 1.1` , and set `Upgrade` and `Connection` headers manually.
